

B U P T

密码工程

第7讲、若干基础问题 (5) 消息认证码、安全信道和实现问题

授课教师：王晨宇

时间：2026年4月16日



目录

contents

B U P T



3.10 消息认证码

3.11 安全信道

3.12 实现问题

3. 10

消息认证码



消息认证码



- ✓ 3.10.1 MAC的作用
- ✓ 3.10.2 理想的 MAC
- ✓ 3.10.3 CBC-MAC
- ✓ 3.10.4 HMAC
- ✓ 3.10.5 MAC的使用



MAC 是什么

□ MAC 是一种函数，它有两个输入：

□ 固定大小的密钥 K ，就像是一把特定的“钥匙”，用来对消息进行特定处理。

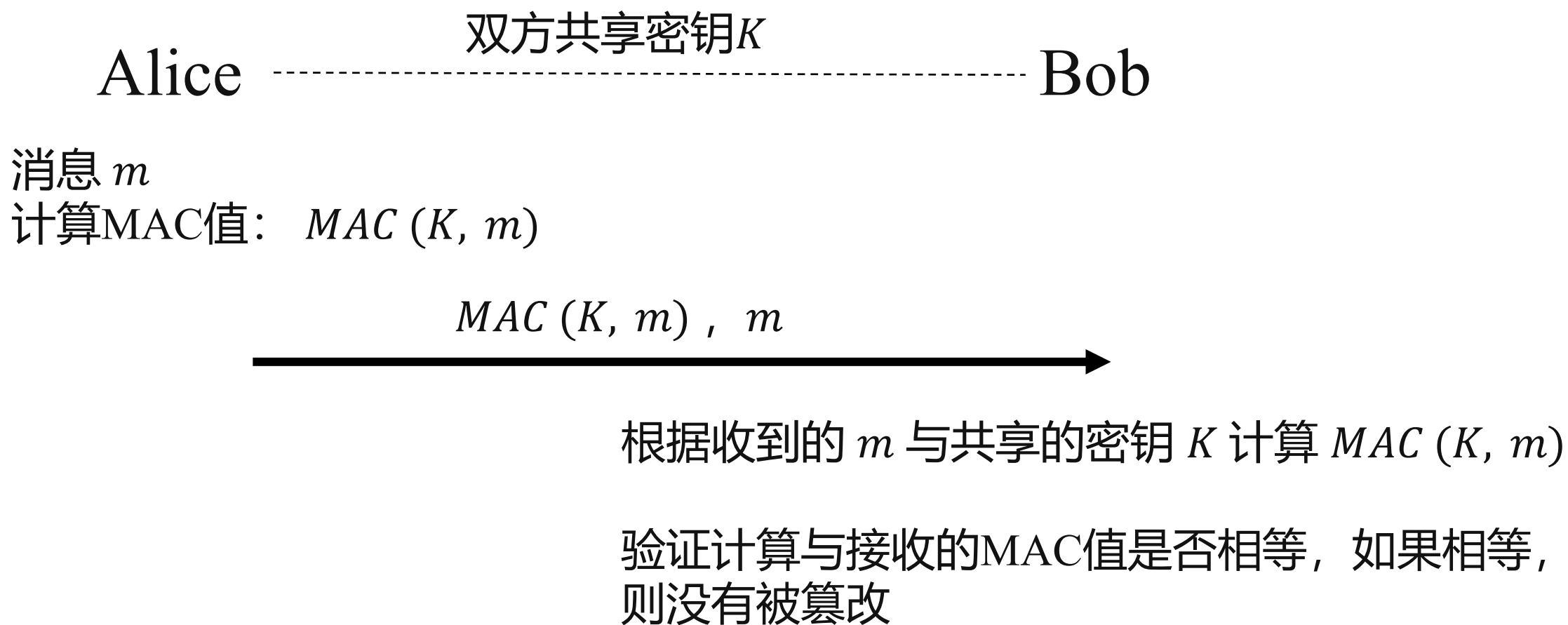
□ 任意大小的消息 m ，也就是我们要传输或者处理的信息内容。

□ 它会产生固定大小的输出，这个输出就是 MAC 值。

□ 可以把 MAC 函数表示为 $MAC(K, m)$ 。



MAC 怎么用于消息认证?





什么是理想的MAC函数

- 理想的 MAC 函数是从所有可能输入到 n 比特输出的随机映射。
- n 代表 MAC 函数输出结果的比特数。

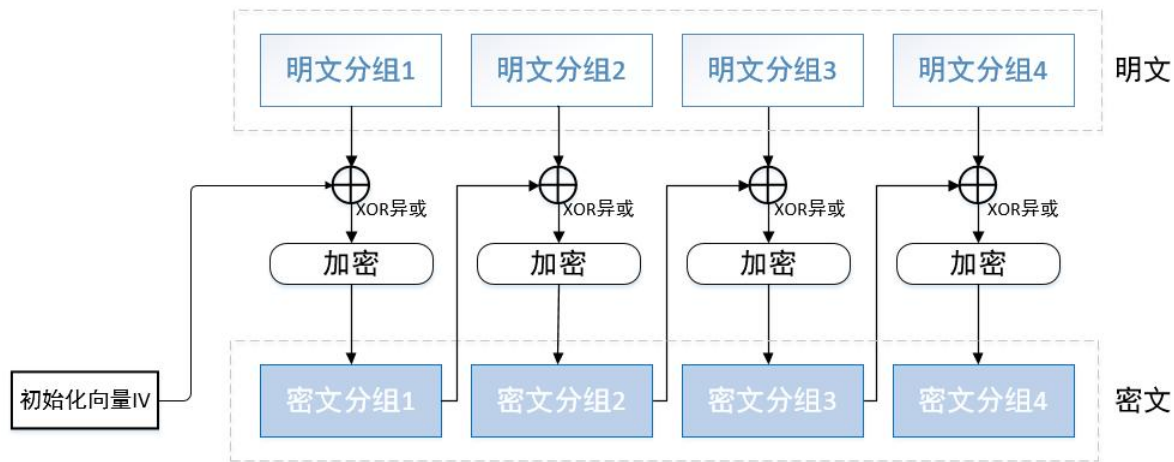
与 Hash 函数异同

- 相同：和理想 Hash 函数一样，都应具有随机映射特性。
- 不同：安全性定义方面，MAC 函数要求更宽泛。



什么是CBC-MAC

- CBC-MAC (Cipher Block Chaining Message Authentication Code) 是一种用于消息认证的加密算法。
- 它通过将消息与密钥 K 进行CBC模式加密, 最后生成一个消息认证码 (MAC) 值, 确保消息的完整性和真实性。



CBC加密模式回顾



CBC-MAC 计算流程

- 初始化向量 (IV) 作为第一个输入。

$$H_0 = IV$$

- 然后，每一块消息与上一步的结果进行异或 (XOR) 运算，并通过加密算法使用密钥 K 进行加密。

$$H_i = E_K(P_i \oplus H_{i-1})$$

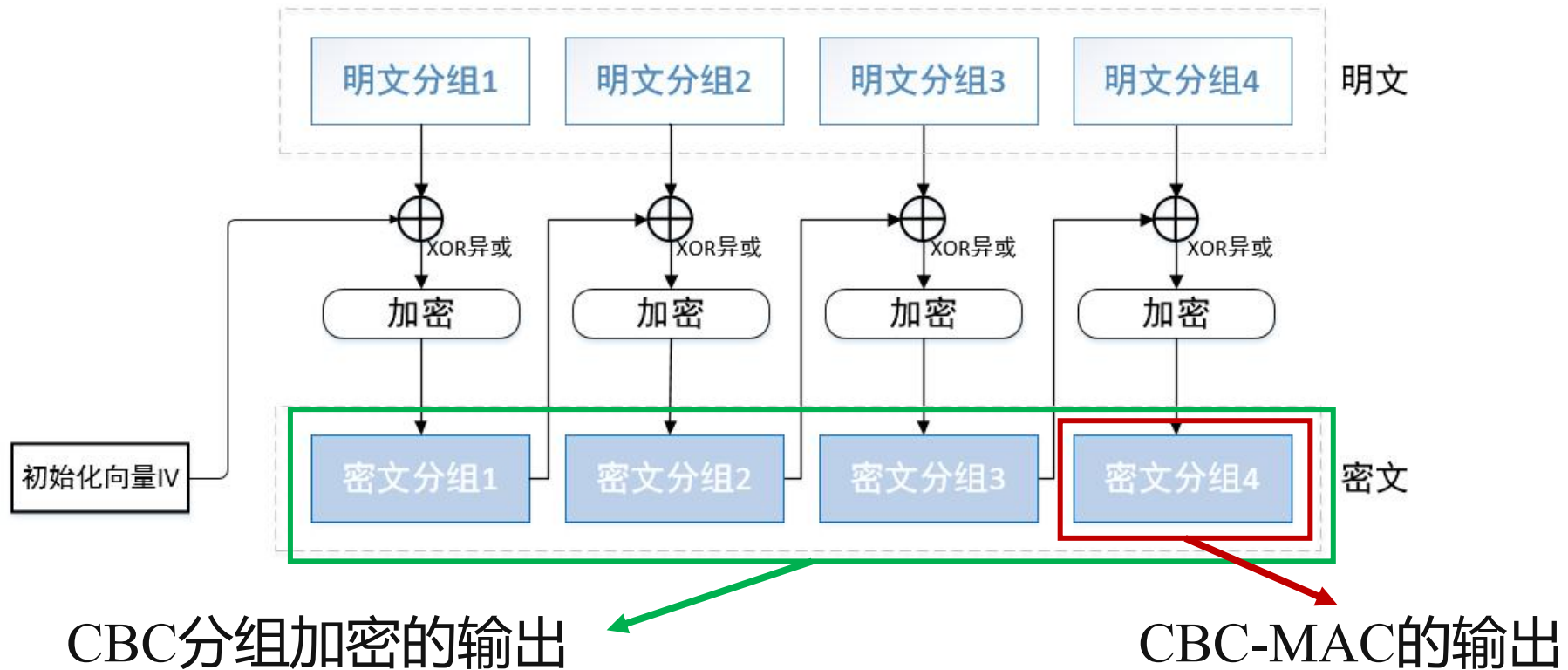
- 最终，经过 k 轮之后，生成的最后一个密文块即为 MAC 值。

$$MAC = H_k$$



CBC-MAC 与 CBC 分组加密模式的区别

- CBC-MAC只取最后一块作为输出
- CBC分组加密则取全部的密文分组





什么是碰撞攻击

- ▣ 碰撞攻击 (Collision Attack) 是一种试图找到两个不同输入，使得它们通过同一个加密算法后，得到相同输出的攻击方式。
- ▣ 攻击者希望通过对消息的篡改，生成一个“伪造”的消息，且系统无法检测到这一篡改。



CBC-MAC 碰撞攻击的流程

- 第一步，攻击者首先收集大量消息的MAC值，知道产生碰撞，即 $M(a) = M(b)$ 。
- 第二步，如果攻击者得到了 $M(a||c)$ 的值，由

$$M(a||c) = E_k(c \oplus M(a))$$

$$M(b||c) = E_k(c \oplus M(b))$$

- 可知，攻击者可以将消息 $a||c$ 替换为 $b||c$ 而无需更改对应的MAC值，同样可以通过接收方的验证。



概念

- HMAC (Hash-based Message Authentication Code) 是一种基于哈希函数的消息认证码 (MAC) 算法, 用于验证消息的完整性和来源的可信性。
- 结合了哈希函数和密钥的概念。
- 例如, 我们可以基于SHA-256实现HMAC:

$$(K, m) \rightarrow SHA-256((K \oplus a) || SHA-256((K \oplus b) || m))$$



HMAC算法流程

□1、密钥填充

□HMAC算法首先对密钥进行预处理，确保它的长度适合哈希函数的要求：

□如果密钥 K 的长度 $>$ 哈希函数的明文分组长度

□先对密钥 K 进行哈希处理。

□如果密钥 K 的长度 $<$ 哈希函数的明文分组长度

□通过填充零使其达到要求的长度。

$$(K, m) \rightarrow SHA-256((K \oplus a) || SHA-256((K \oplus b) || m))$$



HMAC算法流程

□2、创建内外填充常量

□HMAC还会使用到两个固定的填充常量：

□ a （外填充）： $a = 0x5C$

□ b （内填充）： $b = 0x36$

□两个填充常量的长度均与使用的哈希函数的分组长度相同。

$$(K, m) \rightarrow SHA-256((K \oplus a) || SHA-256((K \oplus b) || m))$$



HMAC算法流程

□3、计算HMAC

- 1) 将 K 与 a 进行亦或，得到 K' 。将 K 与 b 进行亦或，得到 K'' ;
- 2) 将 K' 和消息 m 拼接在一起，再通过哈希函数计算得到 $H(K' || m)$ 。
- 3) 代入公式：

$$\begin{aligned} & HMAC(K, m) \\ &= H(K'' || H(K' || m)) \\ &= H(K \oplus b || H((K \oplus a) || m)) \end{aligned}$$



为什么更推荐HMAC?

□ 针对离线攻击的防护:

- HMAC的设计结合了密钥和哈希运算两次，不同于只进行一次哈希运算的普通方案。

- 这样可以有效避免仅用一次哈希运算时所面临的攻击风险。

□ 高效的计算和安全性平衡:

- 与在线攻击相比，离线攻击的实现更加困难。

- 即使攻击者获得了部分信息，由于采用了两次哈希计算，难度也大大提高。



单独的MAC无法验证附加消息

- 在传统的MAC认证中，通常只对消息本身进行认证。
- 假设Alice与Bob交换消息时，若没有确保所有相关信息的认证，攻击者（如Eve）就能进行攻击。
- 例如，Eve可能将Alice发送的消息在错误的时间重新发给Bob，这种情况下，如果只对消息本身进行认证，系统就会无法检测出攻击。



Horton原则

- Horton原则强调，MAC验证的对象应该是消息内容，而不是消息本身。
- 具体而言，为确保认证的完整性，认证时不仅要考虑消息本身，还要考虑消息中的所有相关信息（例如Alice与Bob的标识符，时间戳等）。



Horton原则

□ 例如

- Alice使用MAC对 $m = a||b||c$ 进行认证，其中 a, b, c 为三个数据域。
- Bob在收到 m 并完成认证。
- 但问题是，如何保证Bob也将 m 分成了同样的域，如果其划分规则与Alice的构造不一致，Bob就会得到错误的域值。



1.如果你设计的系统只对消息本身进行 HMAC 验证，而不包含消息的上下文（如发送者、时间戳、协议版本等），请结合 Horton 原则说明该认证方案是否足够安全，会面临哪些风险？

3. 11

安全信道



- ✓ 3.11.1 问题的形式化
- ✓ 3.11.2 认证与加密的次序
- ✓ 3.11.3 解决方案
- ✓ 3.11.4 详细说明



起因

- 通过先前诸多技术的学习，我们又回到了最初所尝试解决的第一个现实问题。
- 即**建立安全的信道**。
- 我们将这个问题定义为在Alice和Bob之间建立安全的连接，接下来我们将逐步对这一问题进行形式化的定义。



背景

- Alice和Bob之间需要建立一个安全的通信信道。
- Alice给Bob发送信息，Bob也会给Alice发送消息。
- 这条信道必须能够抵御外部攻击，且在此信道中传递的信息必须经过验证。
- Eve（攻击者）在这里的任务是监听和篡改Alice和Bob的通信。



密钥的使用

- Alice和Bob共享一个密钥 K ，且不被第三方所知。
- Bob在接收到消息时，使用该密钥即可得知该消息是由掌握密钥 K 的某个参与方发出来的。
- 通信过程中，Bob永远不会将Alice当作一个人来识别，他所识别的仅仅是密钥 K 本身。



对密钥的要求

- 1) 只有Alice和Bob知道密钥 K 。
- 2) 每次安全信道被初始化时，就会基于密钥 K 产生一个新的值。
- **第二条非常重要。**
- 如果相同的密钥反复使用，前一次会话的消息就可以被Alice或Bob重放，这会
引起混乱。
- 因此，即使在Alice与Bob之间已经有一个长期共享的密钥 K ，我们仍然需要在每
次建立安全信道时生成一个临时的会话密钥。



消息的发送流程

□消息发送：

- Alice 和 Bob 通过他们的通信协议来发送消息。
- 为了确保消息的顺序正确，Bob 必须知道消息的顺序，并且没有任何被修改的消息。

□消息的验证：

- 消息的验证不仅仅是 Alice 和 Bob 之间的内容，还包括他们发送和接收的所有信息。

□信道的完整性：

- 信道必须完整，信息传输过程中没有被篡改或修改的地方。



安全性——消息的顺序

- 目标：确保Eve无法获得关于其他消息的任何信息。
- 除了消息大小和发送时间，Eve无法获取其他信息。
- 即使Eve进行主动攻击，即正在传输的消息被修改，Bob也能正确接收到顺序无误的消息。
- Bob接收到的消息顺序是准确的，并且没有被修改的消息。



安全性——确认与重发机制

- 信息确认：Bob确认接收到的信息，可以通过多次重发确保安全。
- Alice会通过重新发送未确认的信息来确保Bob收到。
- 这是为了防止信息丢失或被篡改，确保消息的准确传递。



“先加密再认证”与“先认证后加密”

□ 对消息进行加密和认证，有两种方法：

□ 首先进行加密，然后再对密文进行认证；

□ 首先进行认证，然后再对消息和MAC值进行加密。



先加密再认证

□ **支持理由一：先加密的方案相比于先认证的方案更加安全。**

□ **先认证后加密：**

□ 如果加密方案存在缺陷，安全性容易受到威胁。这是因为认证发生在加密之前，一旦加密存在漏洞，可能无法有效保障信息的安全。

□ **先加密后认证：**

□ 即使加密方案存在缺陷，通过后续认证步骤（如MAC技术）可以检测并确保消息的完整性和安全性，从而避免加密缺陷引发的安全问题。



先加密再认证

- **支持理由二：处理假消息和提高效率。**
- 在正常中，Bob需要对所有消息进行解密和验证。如果Bob接收到大量假消息，则会消耗大量资源先解密再验证。
- 在“先加密再认证”方案中，在解密之前Bob就可以识别这个假消息并直接丢弃。
- 而在“先认证后加密”方案中，Bob必须先进行解密才能确认消息的真假。



先认证后加密

- **支持理由一：认证比加密更重要。**

- **先认证后加密：**

 - MAC的输入和真正的MAC值被隐藏，Eve只能看到密文和加密后的MAC值。

- **先加密后认证：**

 - Eve可以看到MAC的输入和MAC值。

- 如果最后进行的是加密，Eve就设法攻击加密函数；

- 如果最后进行的是认证，Eve就设法攻击认证函数。

- **大多数情况下，对通信数据进行修改所造成的毁坏性远大于看到通信内容。**



先认证后加密

□支持理由二：先认证符合Horton原则

- Horton原则要求对消息内容进行认证，而不是对消息本身进行认证。
- 对于先加密的方案，一个通过Bob认证的密文，可能所使用的密钥并不是Alice进行加密时所使用的密钥。
- 这将会导致Bob所获取的明文并不是Alice所生成的明文。



如何使用消息编号

- Alice将消息分成多个部分，每部分用一个编号标识。
- 每个消息编号都与加密过程相关联。
- 每次加密消息时，消息编号会增加，防止重放攻击。



消息编号的用途

- 1、消息编号保证了每个消息的唯一性，避免重复。
- 2、消息编号可以用来为加密算法提供需要的IV（初始化向量）。
- 3、消息编号能够帮助Bob拒绝重复的消息，并告诉Bob哪些消息丢失了。



认证

- 认证过程需要MAC函数的参与，如前所述，为了遵守Horton原则，MAC函数的计算应如下所示：

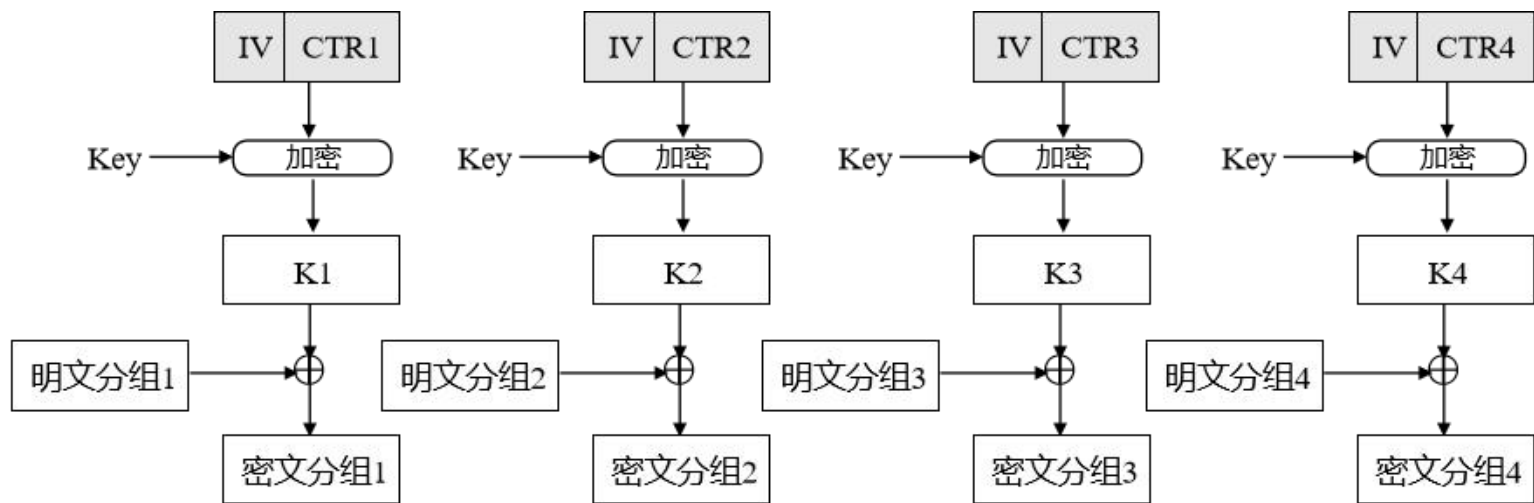
$$MAC(i||l(x_i)||x_i||m_i)$$

- i 为消息编号。
- $l(x_i)$ 则保证了 $i||l(x_i)||x_i||m_i$ 能够唯一分解成它对应的域。
- x_i 为附加的认证数据。
- m_i 为消息。



加密

- 在加密过程中，假设所采用的是CTR模式下的AES，那么消息编号 i 即可为CTR提供唯一的瞬时值，并将之与需要加密的消息 m 、密钥 K 进行组合加密。
- 这样做可以确保每条消息都得到唯一的加密，同时也符合现代的加密标准。



CTR加密模式



格式结构

- 我们不能直接发送加密后的 $m_i || MAC$, 因为Bob也需要得知对应的消息号。
- 实际发送的消息由消息编号 i 与加密的 $m_i || MAC$ 组成, 即

$$(i || m_i || MAC)$$



- 在这一部分，将使用伪代码来描述安全信道。
- 分为：
 - 1) 代码的初始化
 - 2) 发送消息
 - 3) 接收消息



代码的初始化

- 初始化算法有两个主函数：建立密钥和建立消息
- 我们从信道密钥导出4个辅助密钥：
 - Alice给Bob发送消息所用的**加密密钥**和**认证密钥**。
 - Bob给Alice发送消息所用的**加密密钥**和**认证密钥**。

function InitializeSecureChannel

input: ***K*** 256比特的信道密钥

R 角色说明，确定参与方是***Alice***还是***Bob***

output: ***S*** 安全信道的状态



代码的初始化——生成加密密钥和认证密钥

- 首先计算所需要的4个密钥。以第一个为例，意思是生成发送给Bob的**加密密钥**。
 - K是共享的密钥，"Enc Alice to Bob"是一个与加密相关的字符串，合并后进行SHA-256哈希运算，得到加密密钥。
- 其余三个分别是：生成从Bob接收的**加密密钥**、生成给Bob发送的**认证密钥**、生成从Bob接收的**认证密钥**。

$$KeySendEnc \leftarrow SHA_{d-256}(K || \text{"Enc Alice to Bob"})$$

$$KeyRecEnc \leftarrow SHA_{d-256}(K || \text{"Enc Bob to Alice"})$$

$$KeySendAuth \leftarrow SHA_{d-256}(K || \text{"Auth Alice to Bob"})$$

$$KeyRecAuth \leftarrow SHA_{d-256}(K || \text{"Auth Bob to Alice"})$$



代码的初始化——设置发送和接收的计数器

- 将发送和接收的计数器置0。
 - 发送计数器用于记录发送的最后一个消息的编号。
 - 接收计数器用于记录接收的最后一个消息的编号。

$(MsgCntSend, MsgCntRec) \leftarrow (0, 0)$

发送计数器

接收计数器



代码的初始化——状态打包

- 将密钥和计数器信息打包成一个状态信息S并返回。
- 表示初始化过程完成，并且所有的必要数据都已准备好，准备开始进行加密通信。

$$S \leftarrow (KeySendEnc, \\ KeyRecEnc, \\ KeySendAuth, \\ KeyRecAuth, \\ MsgCntSend, \\ MsgCntRnc)$$

return S



发送消息——发送消息的准备阶段

- 这个算法以**会话状态**、**要发送的消息**以及**用于认证的附加数据**为输入，输出准备发送的加密和认证消息。
- 接收者必须拥有相同的附加数据以进行认证检查。

function SendMessAge

input: S 安全会话状态

m 要发送的消息

x 用于认证的附加数据

output: t 发送给接受者的数据



发送消息——更新消息计数器

- 首先，检查 `MsgCntSend` 是否在合法范围内（小于 $2^{32} - 1$ ），如果不在范围内则会抛出异常。
- 更新发送消息的计数器 `MsgCntSend`（发送端计数器）并将其赋值给 i 。
- (i 为本次发送的消息所使用的消息编号)
- 它会根据上次消息编号加 1 来确保每次发送的消息都有唯一编号。

$\text{assert } \text{MsgCntSend} < 2^{32} - 1$

$\text{MsgCntSend} \leftarrow \text{MsgCntSend} + 1$

$i \leftarrow \text{MsgCntSend}$



发送消息——计算认证值

- 使用 HMAC和认证密钥KeySendAuth来进行计算，生成认证值 a 。
- 然后，生成 t ，它结合了密钥、消息计数、消息内容等。

$$a \leftarrow \text{HMAC-SHA-256}(\text{KeySendAuth}, i \parallel l(x) \parallel x \parallel m)$$

$$t \leftarrow m \parallel a$$



发送消息——计算加密密钥流

- 首先获取发送端的加密密钥 ($KeySendEnc$)。这个密钥在初始化阶段已经确定, 用来加密消息。
- 使用密钥流生成算法, 明文块是由4字节的块编号、4字节以及8个全0字节组成
- 最终生成密钥流 k 。

$$K \leftarrow KeySendEnc$$

$$k \leftarrow E_K(0 \parallel i \parallel 0) \parallel E_K(1 \parallel i \parallel 0) \parallel \dots$$



发送消息——生成最终的加密文本

- 这个步骤生成最终的消息 t 。
- 它结合了 i 、 $(m||a)$ 和生成的加密流 k 。

$$t \leftarrow i \parallel (t \oplus \text{First-}l(t)\text{-Bytes}(k))$$
$$\text{return } t$$



接收消息——接收消息的准备阶段

- S : 安全会话状态, 包含密钥、消息计数等信息;
- t : 收到的密文数据 (包含消息编号 + 加密消息 + MAC认证) ;
- x : 用于认证的附加数据 (context, 通常与应用上下文相关) 。

function ReceiveMessage

input: S 安全会话状态

t 接受者收到的数据

x 用于认证的附加数据

output: m 实际发送的消息



接收消息——拆分数据

□ 检查输入长度是否合法，至少要有：

□ 4 字节的消息编号 i

□ 32 字节的认证值 a 。

□ 拆分数据：

□ 将 t 拆分成 $i \parallel t$

$assert\ l(t) \geq 36$

$i \parallel t \leftarrow t$



接收消息——解密获取 m

- 与发送端一样，构造用于解密的密钥流 k 。
- 密文 t 使用密钥流 k 进行异或操作得到明文 m 和 MAC 值 a 。

$$K \leftarrow KeyRecEnc$$

$$k \leftarrow E_k(0 \parallel i \parallel 0) \parallel E_k(1 \parallel i \parallel 0) \parallel \dots$$

$$m \parallel a \leftarrow t \oplus First-l(t)\text{-Bytes}(k)$$



接收消息——重新计算认证

- 使用接收方的认证密钥重新计算认证值，比较 a' 和接收到的 a 。
- 如果认证不通过，说明消息可能被篡改或伪造，直接丢弃。
- 如果认证通过，检查消息编号顺序（防止重放攻击）。
- 随后更新消息编号，返回明文。

```
 $a' \leftarrow \text{HMAC-SHA-256}(\text{KeyRecAuth}, i \parallel l(x) \parallel x \parallel m)$   
if  $a' \neq a$  then  
    destory  $k, m$   
    return AuthenticationFailure  
else if  $i \leq \text{MsgCntRec}$  then  
    destory  $k, m$   
    return MessageOrderError  
fi  
  
 $\text{MsgCntRec} \leftarrow i$   
return  $m$ 
```

3. 12

实现问题



实现问题



- ✓ 3.12.1 创建正确的程序
- ✓ 3.12.2 创作安全的软件
- ✓ 3.12.3 如何保护秘密
- ✓ 3.12.4 代码质量
- ✓ 3.12.5 侧信道攻击



说明书

- 在日常生活中，购买家电或设备都会配一个说明书来说明其功能和用法，对于程序也不例外。**一个好的程序也需要说明文档。**
- 一个好的说明文档应当包括以下三部分：
- **需求说明：**侧重于概述程序功能，而不是详细的实现步骤。
- **功能说明：**详细描述了程序每个模块的外部功能及其作用。
- **实现设计：**描述了程序如何从各个模块的功能中实现所需功能。



测试和修正

- 除了完善的说明文档之外，写出正确程序的第二个问题是“**测试-修正**”。
- 程序员首先写出个程序，然后测试它是否能正确运行。
- 如果不能，则他们就要修正程序的隐错，然后再重新测试。
- 虽然这样也并不能保证一定能得到正确的程序，但可以获得在通常情况下运行正确的程序。



温和的态度

- 计算机领域里的人们对程序中的错误有不可置信的温和态度，于是程序中的错误被认为是自然而然的事情。
- 如果你的文字处理系统突然崩溃，将一天的工作全部丢失，每个人都认为这是正常的，也是可以接受的。
- 通常他们会指责用户说：“你应该时常保存你的工作。”
- 在客户这种温和的态度下，人们就失去了生产正确软件的强烈愿望。



如何着手开发一个好的程序

- 最后一个问题就是：**我们如何着手开始开发一个好的程序。**
- 可以参考航空工业的工程质量控制系统。
- 飞机上的每一个螺母和螺钉在使用之前都必须经过飞行质量的考核认定，任何时候只要机修工带螺丝刀上飞机，他的工作将由管理员进行检查。
- 任何一个操作，都需要检查、记录、事故后要调查原因，并进行修正。



正确的软件 and 安全的软件并不相同

- 功能正确就是正确的软件，但是软件的安全性也非常重要。
- 我们需要确保软件缺乏某个功能，不论攻击者采取什么方法，他都不可能做某事。
- 这样才能避免攻击者通过软件来实现我们本不想让他实现的功能。
- 简单的说：
 - **正确性是确定有某功能，而安全性是确定没有某功能。**
- 而后者的难度非常大。



瞬时秘密

- 对于安全信道，我们有两类秘密：密钥和数据。
- 这两类秘密都是瞬时的秘密，即**不需要长期存储它们**。
- 数据只有在处理每个消息的时候存储，而密钥也仅仅在安全信道期间保存。
- 瞬时秘密保存在计算机的内存里，但是，大多数计算机的内存都并不安全。



清除状态

- 在软件安全领域，确保不再使用的数据被完全删除是非常重要的。
- 对于编写的程序，应该在数据不再需要时立即清除它们。
- 如果没有及时清理，数据可能会被攻击者利用，造成潜在的安全风险。
- 例如
 - 用C语言编写程序，在代码中用到了内存变量，记得在不再需要这些变量时及时清除它们。



清除状态的方法

- 有多种方法可以确保清除数据。
- 一种常见的方法是使用C语言的memset函数，它通过直接清除内存来避免数据残留。
- 这样做不仅能够确保数据不被恢复，还能够优化程序的性能。



交换文件机制的潜在风险

- 当程序在执行时，并不是所有的数据都保存在内存里，一些数据会被保存在磁盘上的交换文件中（即虚拟内存系统）。
- 交换文件用于存储不在内存中的数据，帮助程序继续运行。
- 交换文件可能会被其他程序访问，因此存在数据泄露的风险。
- 如果程序中存储的秘密数据被存入交换文件，即使内存清空，这些秘密数据仍然会存在于磁盘上。



防止交换文件造成的安全问题

- 可以使用系统调用，通知虚拟内存系统使指定的部分内存不被交换。
- 锁定占用的所有内存，确保敏感数据不会被写入交换文件。
 - 但这一办法可能失去操作系统提供的很多服务，如动态内存堆栈。



什么是高速缓冲存储器

- 目前的操作系统，除了主内存，还会配备容量更小但速度更快的高速缓存。
- 高速缓存中保存的是最近使用的数据副本，CPU 会优先从高速缓存中读取数据。
- 如果命中缓存，CPU 可快速读取，否则需从主内存中调入并更新缓存。



高速缓冲存储器带来的风险

- 当我们以为已经清除主内存的秘密信息时，实际上可能只是清除了主内存版本，缓存中的副本还在。
- 这就意味着攻击者可以从缓存中找回被“清除”的秘密信息，存在安全风险。
- 除了清除不彻底，还有一个更隐蔽的风险：
 - 在多核 CPU 系统中，另一个核心可能会修改缓存里的数据，或者读取还未清除的内容。
 - 这种副本本不应该存在，但在现实中却可能一闪而过，造成潜在泄密。



内存重写并不能完全删除数据

- 内存重写操作并不能彻底清除数据。
- 重写数据后，原有数据仍可能部分残留。
- 具体情况因内存类型而异，如 SRAM 和 DRAM。



内存重写并不能完全删除数据

□ SRAM存储特性:

- 有几种情况可以引发这种现象。如果相同的数据存入SRAM(StaticRAM)内存的同一单元一段时间, 那么该数据将变成这个内存单元预置的初始状态。

□ DRAM存储特性:

- DRAM依靠在非常小的电容器上存储一个小电荷来工作, 电容器周围的绝缘材料将会受到它的作用, 导致这些材料发生变化, 特别是会引起杂质迁移。
- 攻击者使用物理方法可以恢复这些数据。



简洁性

- 简洁性是安全性设计中的关键原则。
- 在设计安全系统时，尽量减少选择项，避免给用户过多选择，确保默认选项是安全的。
- 许多系统设计中包含多个加密组件，用户可以选择不同的加密方式，但这可能导致潜在的安全风险。
- 最好的设计方案是提供一个简单明确的选择，确保默认设置是最安全的。



模块化

- 即使已经消除了大量的选项和特性，得到的系统仍然会很复杂。
- 使这样的复杂系统易于管理的技术就是模块化。
- 模块化就是将系统分成一些模块，然后分别设计、分析、实现每一个模块。
- 每一个模块都应该只解决自己的问题，应该尽量避免其能解决其他模块的问题。



声明

- 声明是改善代码质量的有用工具。
- 当编写密码系统的代码时，每个模块都不能信任其他模块，总是要检测参数的有效性，并且拒绝执行不安全的操作，这大多都是通过声明来进行的。
- 如果模块声明要求在使用一个对象前必须进行初始化，那么在初始化之前使用它就会产生一个声明错误。
- **一个程序所能做的最坏的事情就是产生错误的答案，但若至少能让用户知道出现了错误，这样的程序就还可以接受。**



测试

- 广泛的测试总是研发过程的一部分，它可以帮助我们发现程序中的隐错，但对发现安全漏洞却没有什么用。
- 千万不要将测试与安全性分析混淆，这两者互相补充，但完全不同。



测试的分类

□功能说明测试：

- 适用于相对简单的模块，测试目标是确保模块按功能要求正常工作。

□开发者自测：

- 适用于更复杂的模块，用于验证程序实现的正确性。
- 通常涉及到更复杂的边界条件和特殊情况，例如缓存、随机数生成等。



实现问题

3.12.5 侧信道攻击

- 有一类攻击我们称之为侧信道攻击(side-channel attack)。
- 例如，攻击者可以详细地估量系统加密一条消息所需的时间。
- 现在的功率分析技术完全可以攻击过去的智能卡。对嵌入智能卡的密码系统，攻击者还可以估计智能卡需要的电流。
- 在现实生活中，情况并没有这么糟，因为大多数侧信道攻击都难以实现，但仍需采取一些预防措施。



2.假设你开发了一个安全软件，某变量保存了用户的临时私钥信息。当用户完成操作退出程序后，这段内存变量还保留在系统中。

- 1) 这种实现可能带来哪些安全隐患？**
- 2) 你会采用什么样的策略来规避这个问题？请简要说明实现方式。**



课后作业

B U P T

- 1.假设需要加密的明文信息为 $m=85$ ，选择： $e=7$ ， $p=11$ ， $q=13$ ，说明使用RSA算法的加密和解密
- 2.假设你准备用 C/C++ 实现一个带有 HMAC 验证的简单消息发送模块。请简要描述你会在消息结构中包含哪些字段，并说明各字段的作用。

谢谢大家
